АПТЕЧКА К ЛАБОРАТОРНОЙ РАБОТЕ №3
ТИПИС

Помехоустойчивость 2
Общие принципы помехоустойчивого кодирования 2
Классификация помехоустойчивых кодов 3
Характеристики блочных линейных кодов 5
Про переходы и задачи помехоустойчивого кодирования 6
Граница Хемминга 8
Граница Варшамова-Гилберта 9
Матричные коды. Канонический код 11
Матричные коды. Код Хемминга 22
Циклические коды. Основная теория 27
Кодирование циклическим кодом 31

Помехоустойчивость

Помехоустойчивость – способность ИС противостоять воздействию помех.

Помехоустойчивыми (корректирующими) называются коды, позволяющие обнаружить и при необходимости исправить ошибки в принятом сообщении.

Общие принципы помехоустойчивого кодирования

1. Использование избыточности. Например, в виде дополнительных символов.
2. Свойство усреднения, означающее, что избыточные символы зависят от нескольких информационных символов. Информация, содержащаяся в исходной кодовой последовательности, перераспределяется на избыточные символы.
3. M – мощность первичного алфавита = количество знаков в нем. Зная мощность, можно знать количество битовых разрядов для кодирования сообщений.

$$K \geq \log_2 M$$

Общее число кодовых комб.

$$S_p = 2^K, \quad S_p \geq M$$

После первичного кодирования производится вторичное кодирование, в ходе которого к информационным K-разрядам добавляются R-проверочных разрядов (корректирующих).

Число возможных комбинаций из N-разрядов:

S = 2 ^ N

Количество запрещенных комбинаций – Sf = S – Sразр.

Классификация помехоустойчивых кодов



Рис. 1. Классы

Корректирующие коды делятся на **блочные** и **непрерывные** коды. И те, и другие подразделяются на **разделимые и неразделимые**. **Разделимые** делятся на **систематические и несистематические**. Неразделимые делятся на коды **с постоянным весом и с переменным весом.**

Систематические делятся на циклические коды и матричные. Матричные делятся на канонические и коды Хемминга.



Непрерывные коды (цепные, сверточные, рекуррентные) представляют собой непрерывную последовательность бит, не разделяемую на блоки (информационные и проверочные биты чередуются по определенному правилу).

В блочных кодах информационные и проверочные сгруппированы в блоки. Разделимыми называются коды, в которых информационные и проверочные биты располагаются в строго определенных позициях.

В неразделимых кодах определенности нет.

Коды с постоянным весом – коды, у которых соотношение информационных и проверочных разрядов не изменяется в процессе кодирования.

В кодах с переменным весом такого качества нет.

Систематические (линейные) коды – коды, в которых проверочные биты являются результатом линейных операций над информационными разрядами.

В несистематических кодах информационные и проверочные биты либо вообще не имеют связи, либо эта связь нелинейна.

Характеристики блочных линейных кодов

- Значность (длина) – количество разрядов в каждой кодовой комбинации. Обозначается буквой n.
- Вес (**omega w**) - количество единиц в кодовой комбинации.
- Кодовое расстояние **d** (расстояние Хемминга) – количество разрядов, в которых комбинации отличаются одна от другой. Степени различия двух кодовых комбинаций. Применяется только для кодовых слов одинаковой длины.



Рис. 2. Пример использования

Но также можно проксорить каждый разряд с каждым и взять от этого вес. Попробуем на нашей задачке:

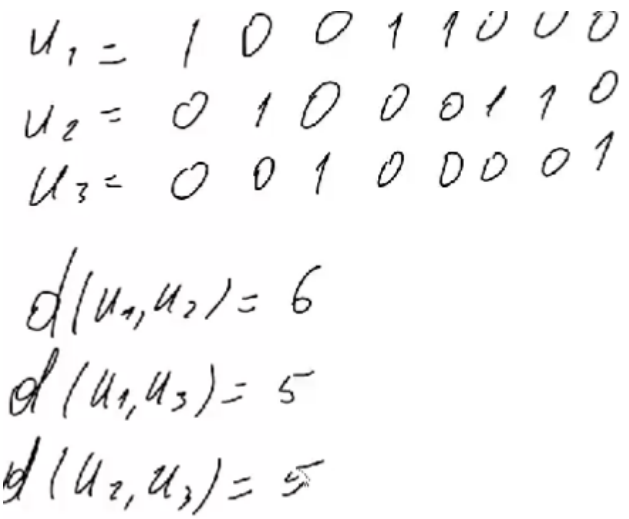


Рис. 3. Для задачи про разведчиков

Чем больше кодовое расстояние, тем выше вероятность исправления ошибки.

Когда в кодовых комбинациях возникает ошибка типа «инверсия», количество ошибочных (искаженных) разрядов будет называться кратностью ошибки.

Однократная ошибка – инверсия только одного разряда из всей кодовой комбинации.

Двукратная – двух разрядов. И так далее.

*Вектор ошибки **e** – двоичная последовательность длиной n с единицами в тех позициях, в которых произошли ошибки. Вес вектора ошибки равен кратности ошибки.*

Про переходы и задачи помехоустойчивого кодирования

Могут быть переходы, которые нас устраивают и не устраивают. Может разрешенный перейти в запрещенный, и мы однозначно скажем, что такого кода быть не может.

Но бывает, что один разрешенный перейдет в другой разрешенный. Тут уже мы вряд ли что-то поймем, где и какая случилась проблема. Задача помехоустойчивого кодирования: уменьшить вероятность того, чтобы символы переходили в другие. В идеальном случае – каждому запрещенному разряду соответствует только один разрешенный.

Но чаще всего ситуация выглядит совсем иначе

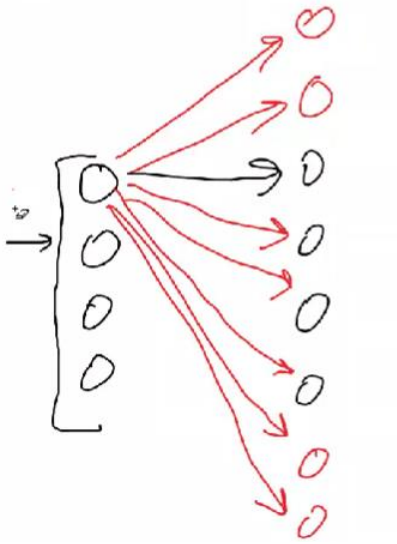


Рис. 1. Как происходит в реальной жизни

Доля обнаруживаемых ошибок – ***Sраз * S. Это всевозможные ошибки. Sзапр * Sразр это количество переходов в запрещенные комбинации.***

Для данного случая: 16/32 = 1/2. Такую долю ошибок мы можем обнаружить.

$$\frac{S_1 * S_2}{S_1 \times S} = \frac{S_1 (S - S_1)}{S_1 * S} =$$

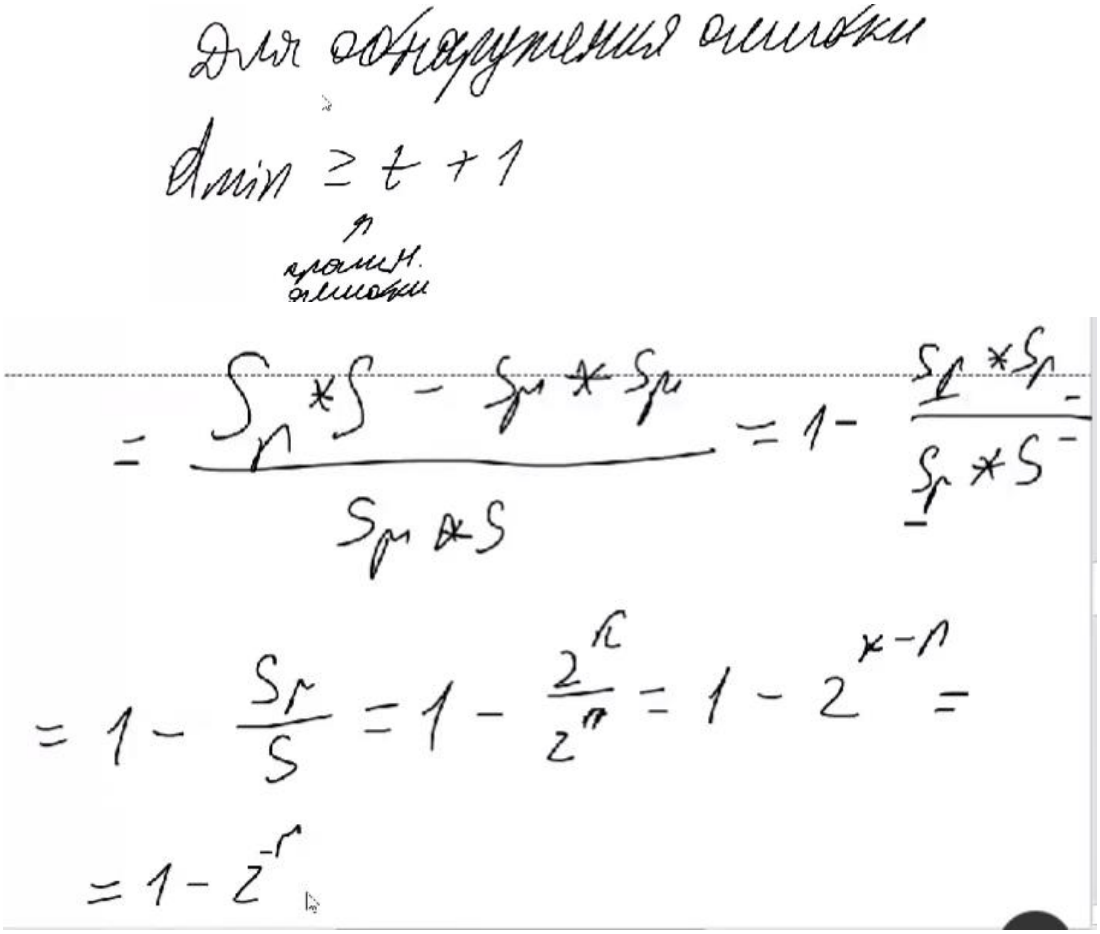


Рис. 2. Формула

Доля исправляемых ошибок – мы можем исправлять только в том случае, если у нас какая-то одна разрешенная комбинация переходит в одну запрещенную.

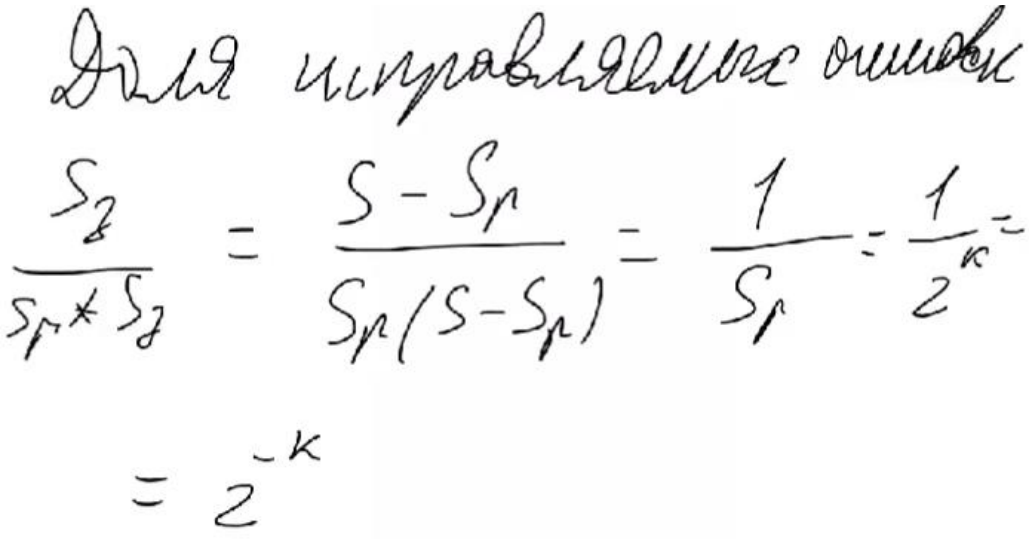


Рис. 3. Формула

Доля зависит только лишь от числа разрешенных комбинаций

Граница Хемминга

Граница Хемминга позволяет определить минимальное число проверочных разрядов R, при котором **теоретически** может быть обеспечена требуемая корректирующая способность (исправление ошибки кратности до t включительно).

$$2^n - 2^k \geq E * 2^k$$

$$\frac{2^n}{2^k} - 1 \geq E$$

$$\frac{2^n}{2^k} \geq E + 1$$

$$2^r \geq 1 + \sum_{i=1}^{t} C_n^i \qquad C_n^0 = 1$$

$$2^r \geq \sum_{i=0}^{t} C_n^i \equiv r \geq \log_2 \left(\sum_{i=0}^{t} C_n^i \right)$$

граница Хемминга

Граница Варшамова-Гилберта

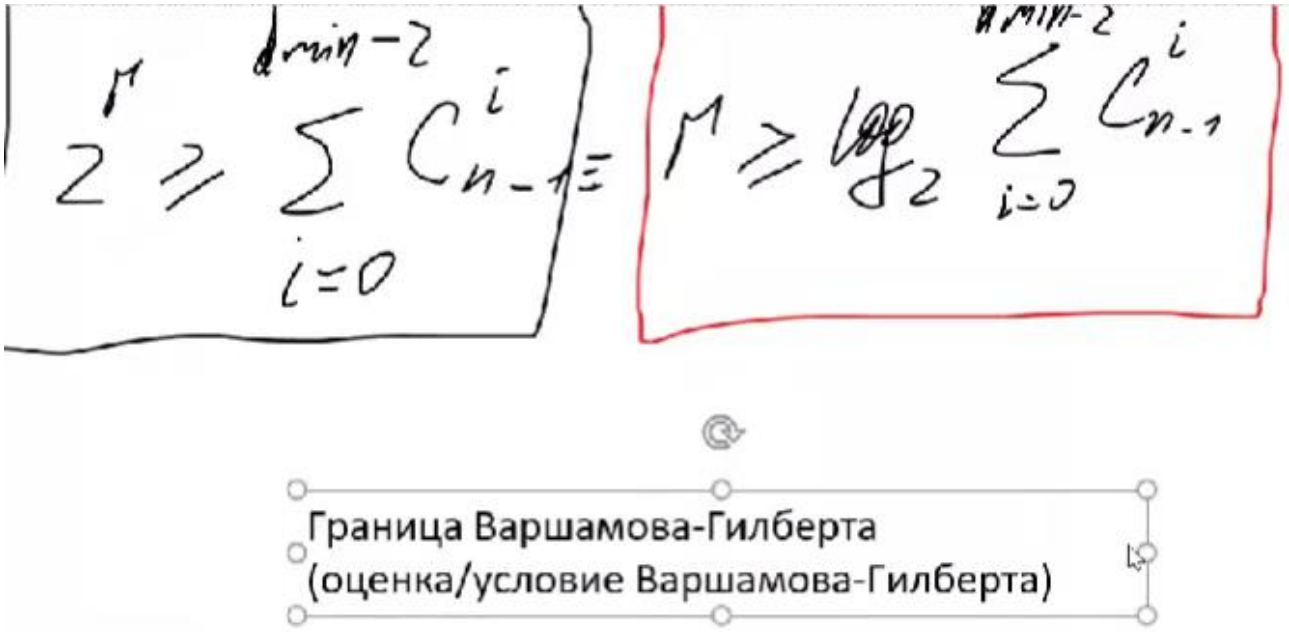


Рис. 1. Формула Варшамова-Гилберта

Позволяет определить минимальное число проверочных разрядов R, при котором **гарантированно** может быть обеспечена требуемая корректирующая способность (исправление ошибки кратности до t включительно).

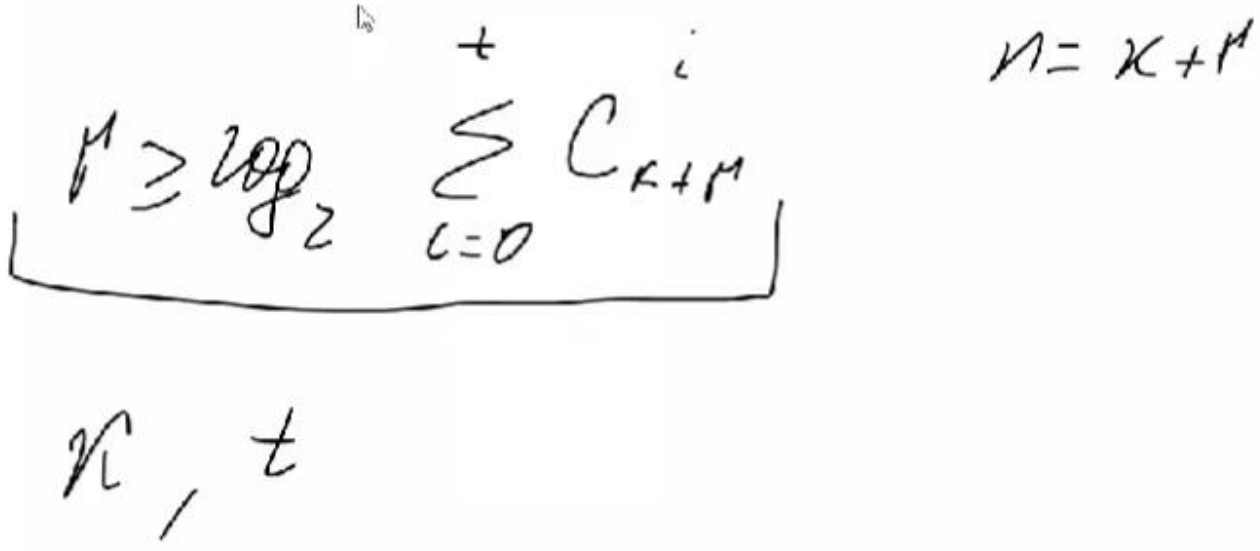


Рис. 2. Выраженная формула

Затем методом подбора находим R.

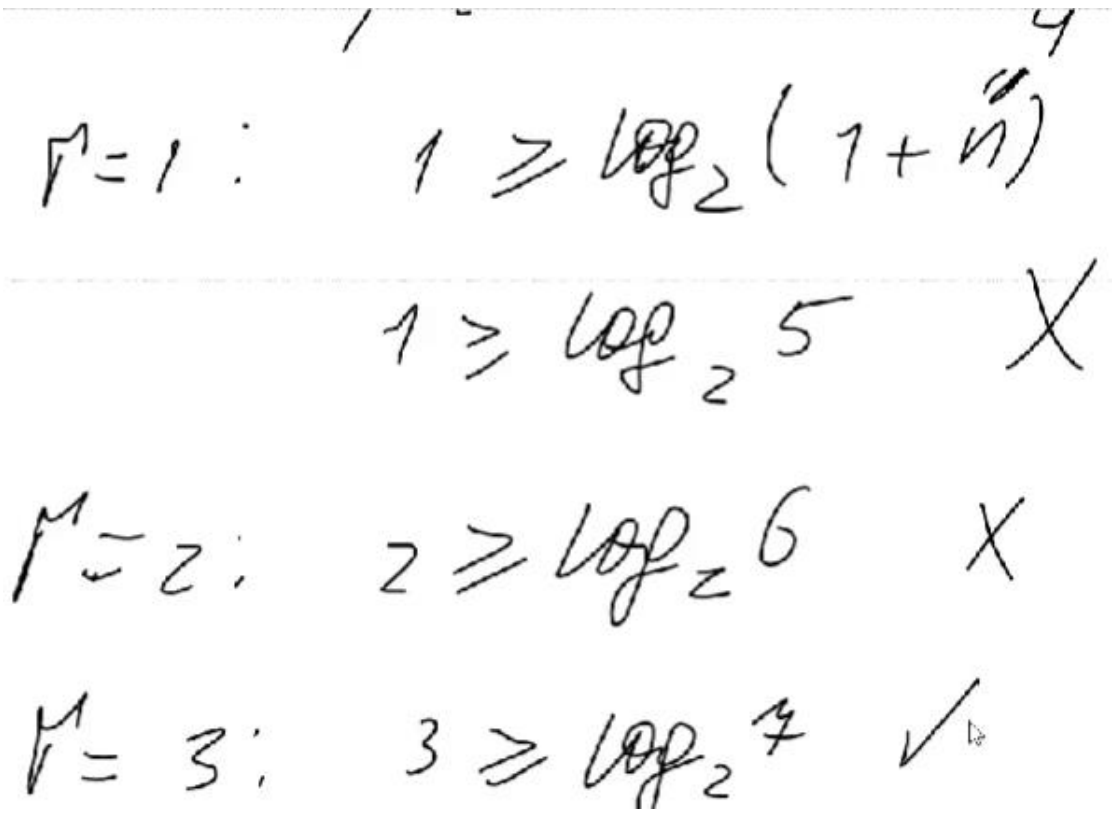


Рис. 3. Метод подбора

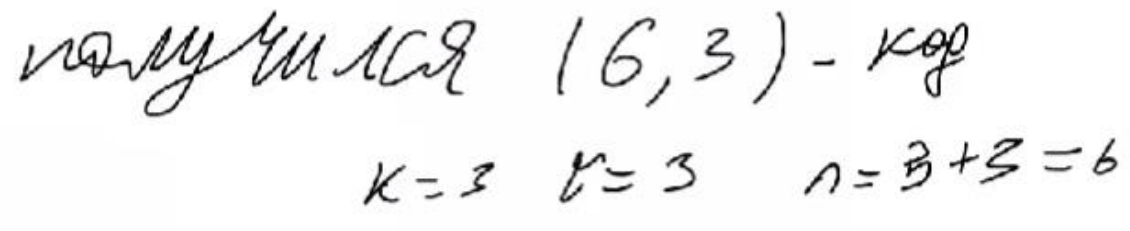


Рис. 4. Результат

Причем r -> min.

Матричные коды. Канонический код

Все разрешенные комбинации матричного (n, k)-кода можно получить имея k-значные исходные комбинации.

При этом:

1. В число исходных комбинаций не должна входить нулевая (тривиальная)
2. Все кодовые комбинации должны быть линейно-независимы, т.е. ни одна из них не может быть получена путём суммирования других.
3. Для обеспечения требуемой корректирующей способности минимальное кодовое расстояние Dmin кодовых комбинаций должно удовлетворять условиям Хемминга (или другим границам)

Для получения кодовых комбинаций используется порождающая матрица, которая состоит из k-строк и n-столбцов. K – количество информационных разрядов, N – разрядов всего.

$$G = \left[\begin{array}{cccccccc} a_{11} & a_{12} & \dots & a_{1k} & b_{11} & b_{12} & \dots & b_{1r} \\ a_{21} & a_{22} & \dots & a_{2k} & b_{21} & b_{22} & \dots & b_{2r} \\ \hline a_{k1} & a_{k2} & \dots & a_{kk} & b_{k1} & b_{k2} & \dots & b_{kr} \end{array}\right]$$

Рис. 5. Порождающая матрица

Канонический вид порождающей матрицы:

Первые k столбцов предназначены для информационных разрядов, оставшиеся для проверочных.

$$G = [I_k \mid P]$$

$$I_k = \left[\begin{array}{cccc} a_{11} & a_{12} & \dots & a_{1k} \\ a_{21} & a_{22} & \dots & a_{2k} \\ \hline a_{k1} & a_{k2} & \dots & a_{kk} \end{array}\right] \qquad P = \left[\begin{array}{cccc} b_{11} & b_{12} & \dots & b_{1r} \\ b_{21} & b_{22} & \dots & b_{2r} \\ \hline b_{k1} & b_{k2} & \dots & b_{kr} \end{array}\right]$$

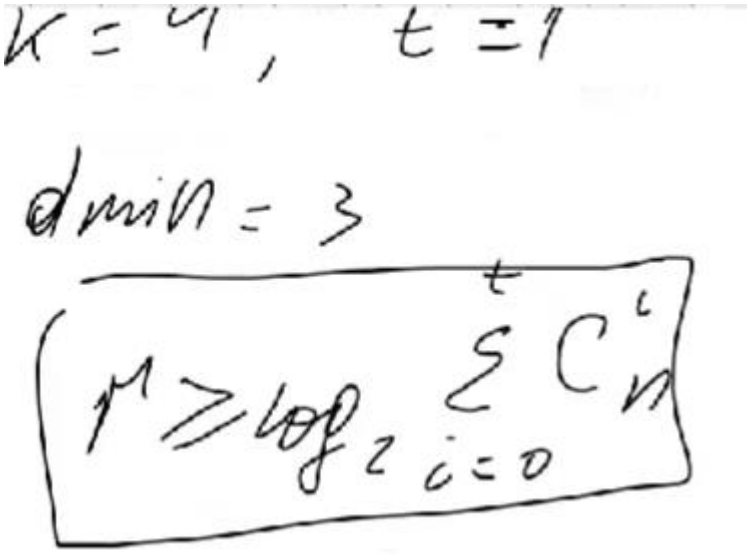
Матрицу G можно представить в виде двух подматриц: информационной Ik и проверочной P.

Обычно в качестве информационной берут квадратную единичную матрицу:

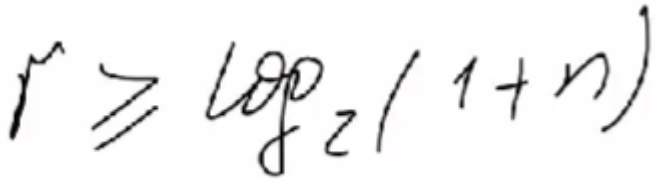
$$I_k = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \hline 0 & 0 & \dots & 1 \end{bmatrix}$$

А вот проверочная матрица P строится с соблюдением нескольких условий (у нас их 4, а в книжке есть лишь 3 из них).

1. Все строки подматрицы P должны быть различны.
2. Кодовое расстояние между двумя любыми строками подматрицы P должно быть не менее $d_{min} - 2$, либо кодовое расстояние между любыми двумя строками G не менее d_{min}.
3. Строки подматрицы P должны быть линейно-независимыми, т.е. ни одна из них не должна являться суммой других.
4. *** (Если будут ошибки в проверочных разрядах). Вес каждой строки подматрицы P должен быть не менее, чем $d_{min} - 1$. Данное правило необходимо для того, чтобы корректно обнаруживать ошибки не только в информационных разрядах, но и в проверочных.



И вообще если t = 1; частный случай:



Допустим, r = 1. Тогда правая часть n = 4 + 1 = 5. $Log_2 6 > 2$.

$$r \geq \log_2(1+n)$$

$r = 1 \quad n = 4+1 = 5 \quad \log_2(1+5)$ ✗

$r = 2 \quad n = 6 \quad \log_2(1+6)$ ✗

$r = 3 \quad n = 7 \quad \log_2(1+7)$ ✓

$\log_2(1+7) = 3$

У нас получился (7, 4) – код. Пример в книге.

Пример построения G

K = 5 – количество информационных разрядов

T = 1 – кратность ошибки

Мы должны построить порождающий код, чтобы мы могли исправить эту ошибку в дальнейшем.

$r = 3 \quad n = 5+3 = 8 \quad \log_2(1+8)$ ✗

$r = 4 \quad n = 9 \quad \log_2(10)$ ✓

$$d_{min} = 2t+1$$

$$= 2 \cdot 1 + 1 = 3$$

$$d_{min} - 2 = \boxed{1}$$

Теперь нам надо построить порождающую матрицу, имея (9, 5) – код.

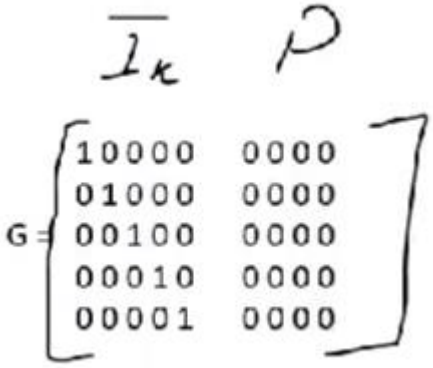
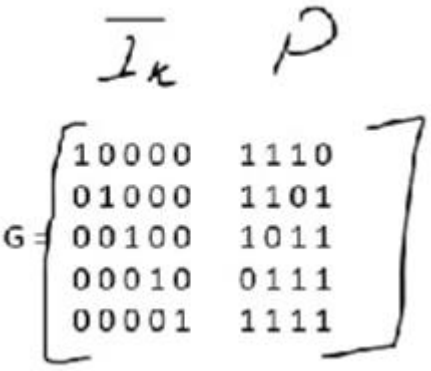


Рис. 6. Заготовка



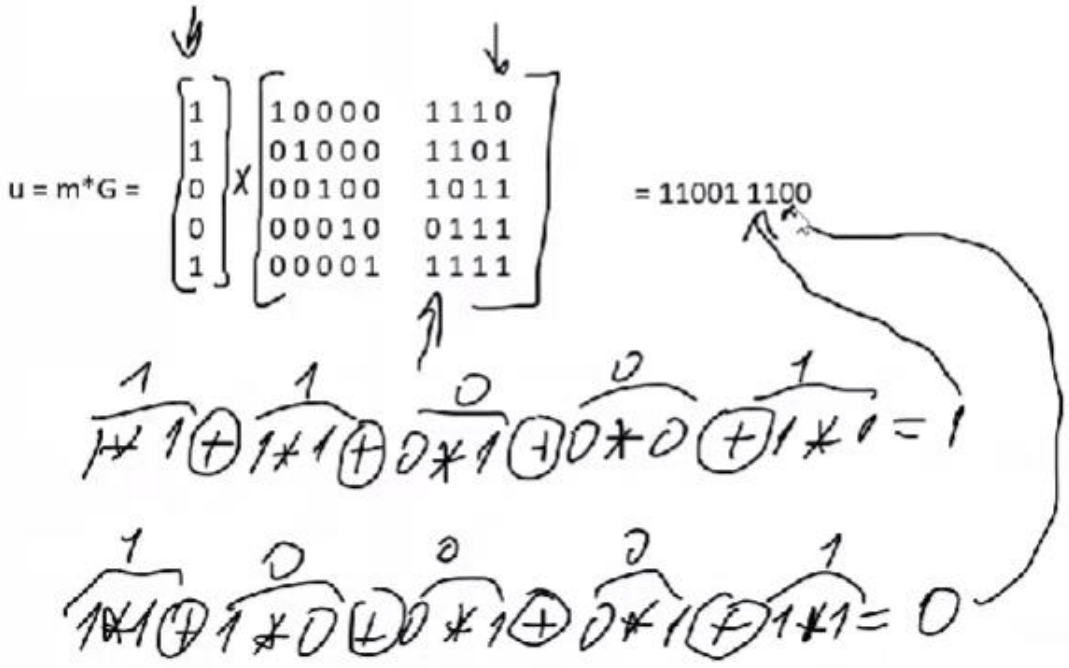
Можно проверить, что во всех случаях, кроме последней, кодовое расстояние равно двум. Вот мы построили нашу порождающую матрицу. Как с помощью неё что-либо закодировать?

Всё также как и всегда при умножении матриц, но всё сложение будет по модулю 2. (XOR)



Рис. 7. Кодирование

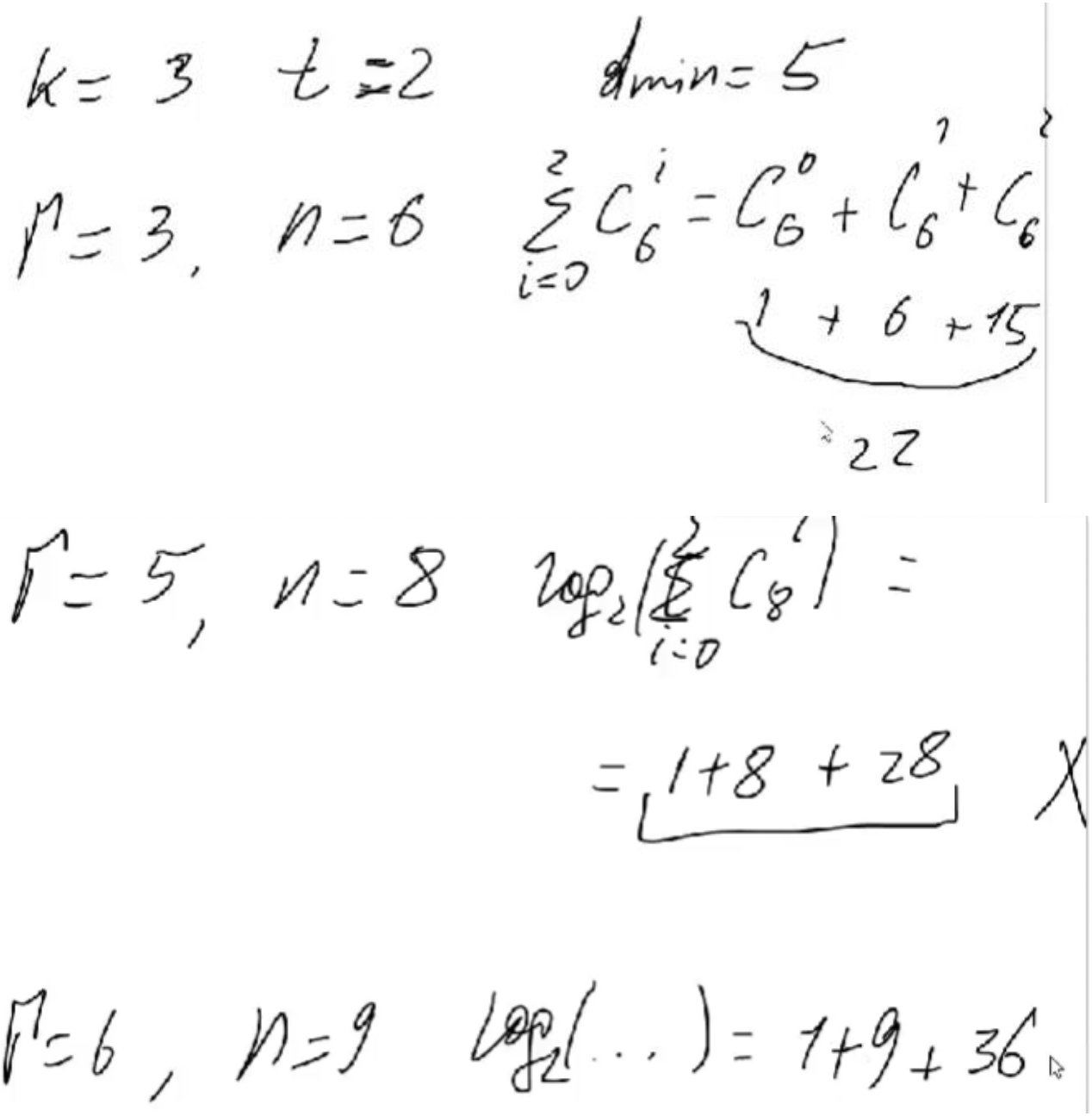
Можно пользоваться правилом, что если четное количество единиц – то ноль, поскольку у нас сложение по модулю 2.



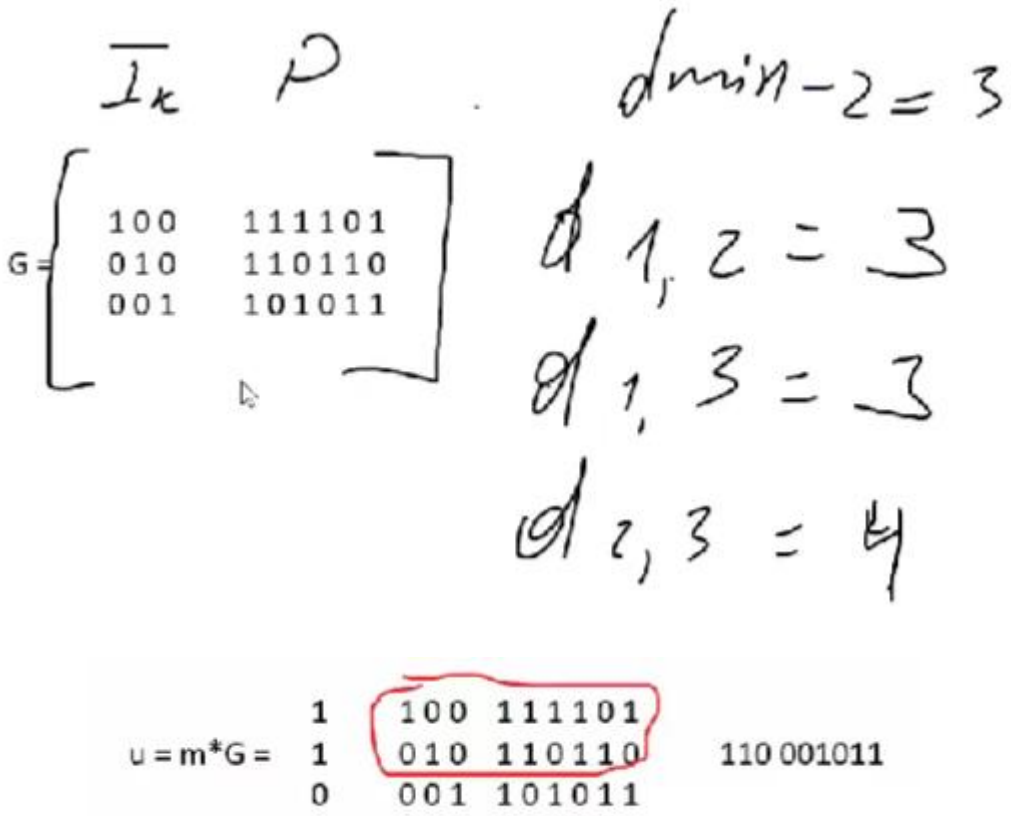
k = 3

t = 2 (кратность ошибки 2)

$d_{min} = 5$



Шесть разрядов — это то, что нам нужно. В итоге у нас есть (9, 3)-код для построения порождающей матрицы.



Обнаружение ошибок с помощью

Для исправления ошибок потребуется проверочная матрица H, которую строят из порождающей матрицы G.

Проверочная матрица должна быть ортогональна любой разрешенной комбинации (при умножении разрешенной комбинации на H, должен получаться нулевой вектор).

Алгоритм построения порождающей матрицы:

1. В порождающей матрице G выделяют информационную подматрицу I_k и проверочную матрицу P.
2. Проверочную подматрицу P транспонируют, получая подматрицу P^T.

$$P=\begin{bmatrix} b_{11} & b_{12} & \dots & b_{1r} \\ b_{21} & b_{22} & \dots & b_{2r} \\ \hline b_{k1} & b_{k2} & \dots & b_{kr} \end{bmatrix} \qquad P^T=\begin{bmatrix} b_{11} & b_{21} & \dots & b_{k1} \\ b_{12} & b_{22} & \dots & b_{k2} \\ \hline b_{1r} & b_{2r} & \dots & b_{kr} \end{bmatrix}$$

3. Справа к подматрице P^T приписывают единичную матрицу I_r, размера: r * r.

$$H = [P^T \mid I_r]$$

ПРИМЕР

$$G = \begin{matrix} 10000 & 1110 \\ 01000 & 1101 \\ 00100 & 1011 \\ 00010 & 0111 \\ 00001 & 1111 \end{matrix}$$

$$P = \begin{matrix} 1110 \\ 1101 \\ 1011 \\ 0111 \\ 1111 \end{matrix} \qquad P^T = \begin{matrix} 11101 \\ 11011 \\ 10111 \\ 01111 \end{matrix}$$

$$H = \begin{matrix} 11101 & 1000 \\ 11011 & 0100 \\ 10111 & 0010 \\ 01111 & 0001 \end{matrix}$$

Рис. 8. Пример с первой матрицей

$$H = \begin{matrix} 11101 & 1000 \\ 11011 & 0100 \\ 10111 & 0010 \\ \underbrace{01111}_{P^T} & \underbrace{0001}_{I_r} \end{matrix}$$

Матрицу H используют для обнаружения ошибок путём проверок ортогональности матрицы H полученному кодовому слову ~u.

Вычисляют вектор s (синдром ошибки), который показывает

S = H * ~u.

Если S != 0:

В ~u присутствует ошибка и её можно исправить.

Синдром ошибки s однозначно определяется вектором ошибки **e.**

u = 110011100

e = 001000000

~u = 111011100

$\tilde{u} = u \oplus e$

$s = H * \sim u = H * (u (+) e) = H * u (+) H * e = H * e$ *(поскольку* $H * u = 0$*)*

$s = H * e$

$1*1 \oplus 1*0 \oplus 1*0 \oplus \ldots$

e = 100000000

$$H = \begin{matrix} 111011000 \\ 110110100 \\ 101110010 \\ 011110001 \end{matrix} \qquad s = \begin{matrix} 1 \\ 1 \\ 1 \\ 0 \end{matrix}$$

$S = H * e$

$S = H * \tilde{u}$

~u = 111011100

$$H = \begin{matrix} 111011000 \\ 110110100 \\ 101110010 \\ 011110001 \end{matrix} \qquad S = \begin{matrix} 1 \\ 0 \\ 1 \\ 1 \end{matrix}$$

u = 110011100

m = 11001

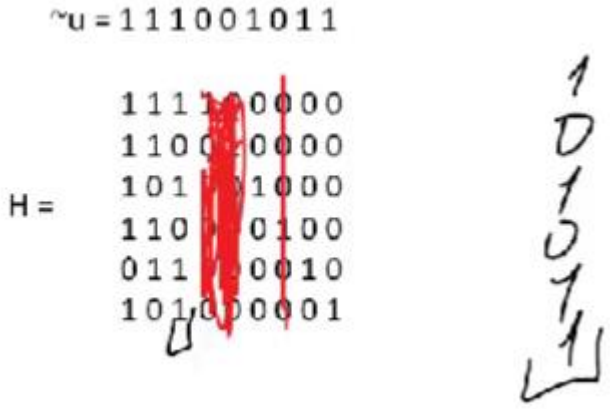
к шифр-ту

Значит ошибка произошла в третьем столбце. (см. Рисунок выше)

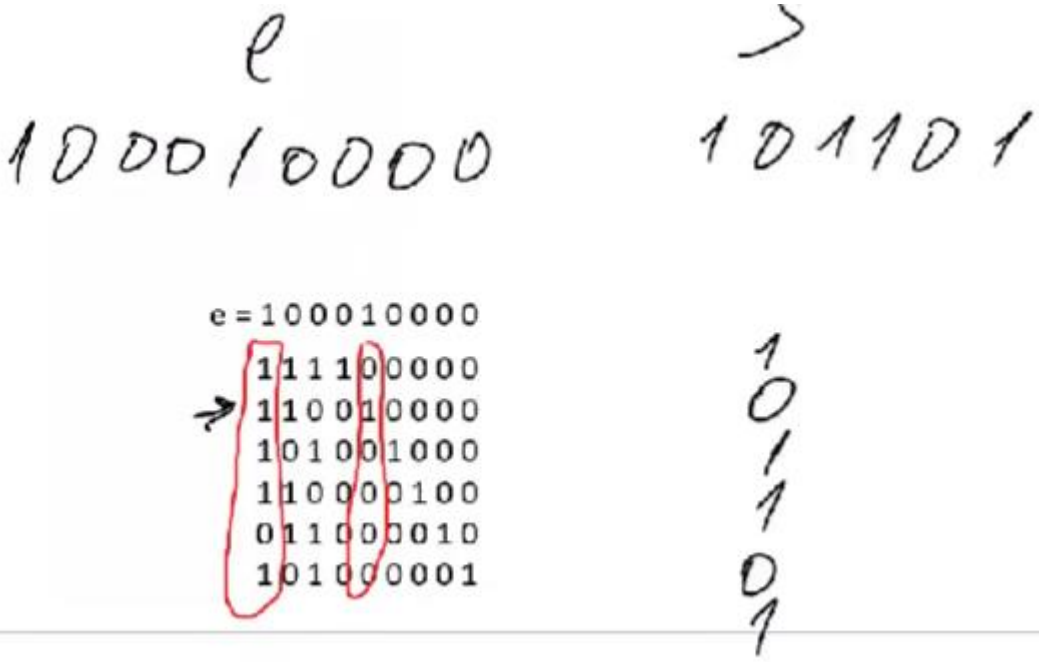
Пример для t = 2



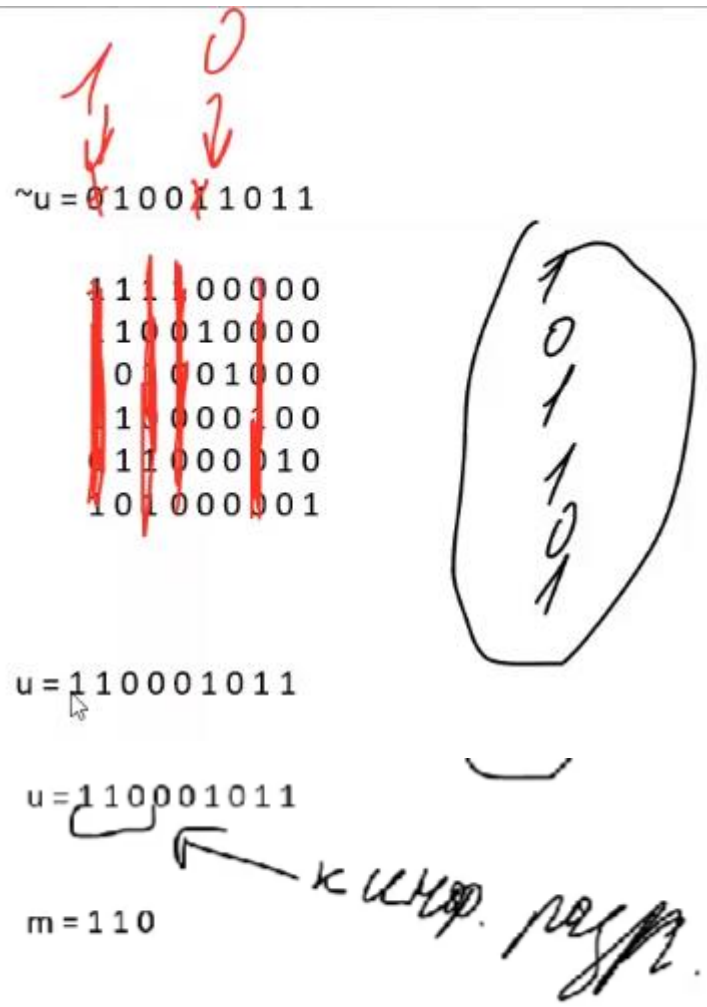
Рис. 9. Однократная и двукратная ошибки



Получился третий столбец, значит в нём и ошибка.

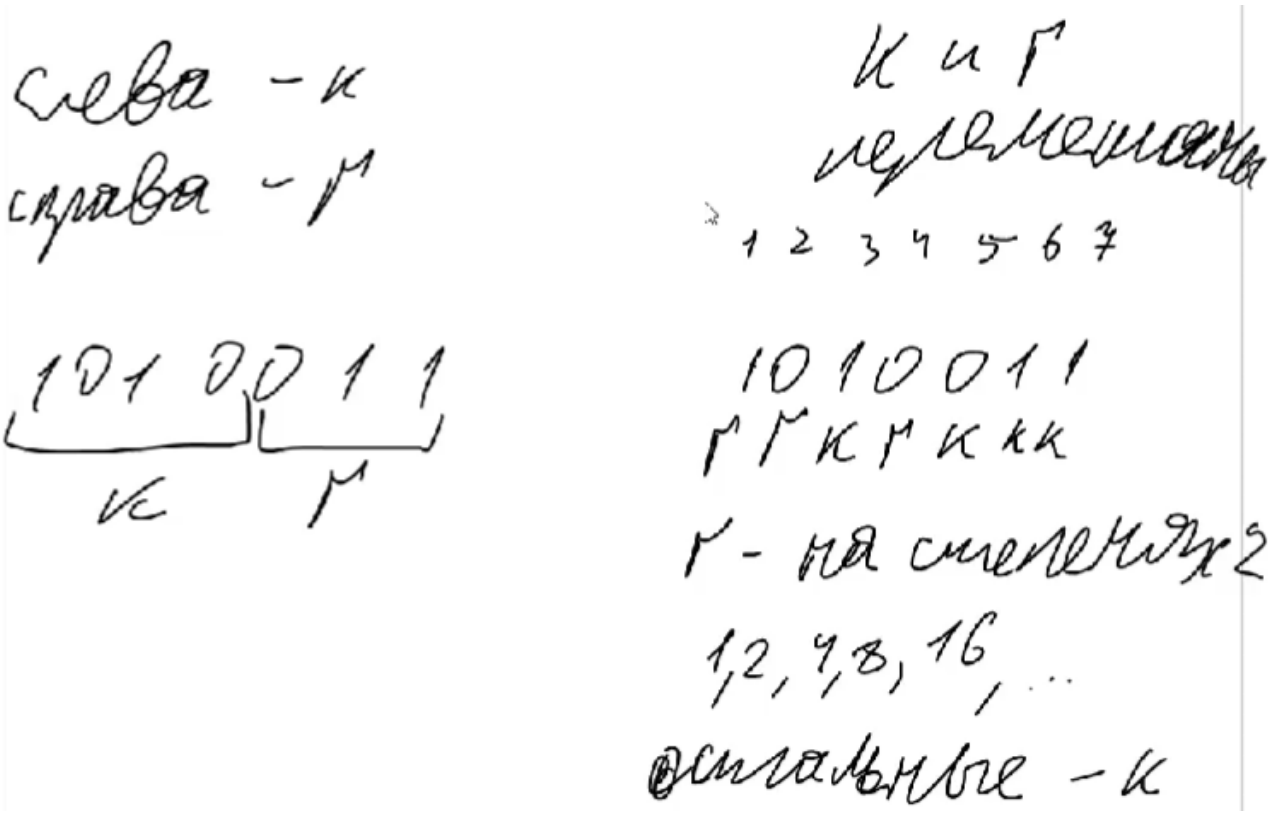


Теперь найдем синдром от умножения второй ~u на H:


~u = 0 1 0 0 1 1 0 1 1
u = 1 1 0 0 0 1 0 1 1
u = 1 1 0 0 0 1 0 1 1
m = 1 1 0

Матричные коды. Код Хемминга

Позволяет очень быстро и легко для исправления однократной ошибки. В каноническом коде слева K разрядов, справа R разрядов. Сначала идут информационные, потом проверочные. В коде Хемминга они перемешаны. Проверочные находятся на битах со степенями двойки.

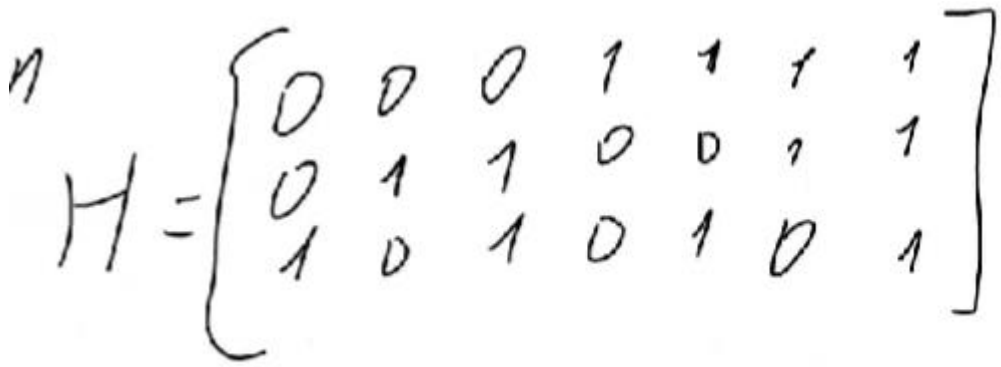


Остальные – k разрядов. Для чего перемешивали? Чтобы локализация ошибки была более простой и более быстрой, чем сравнивать с таблицей синдромов. Код Хемминга предназначен для ошибки кратности 1 (t = 1). Изобрели специальный алгоритм построения кода Хемминга:

1) Пример 7:4 кода:

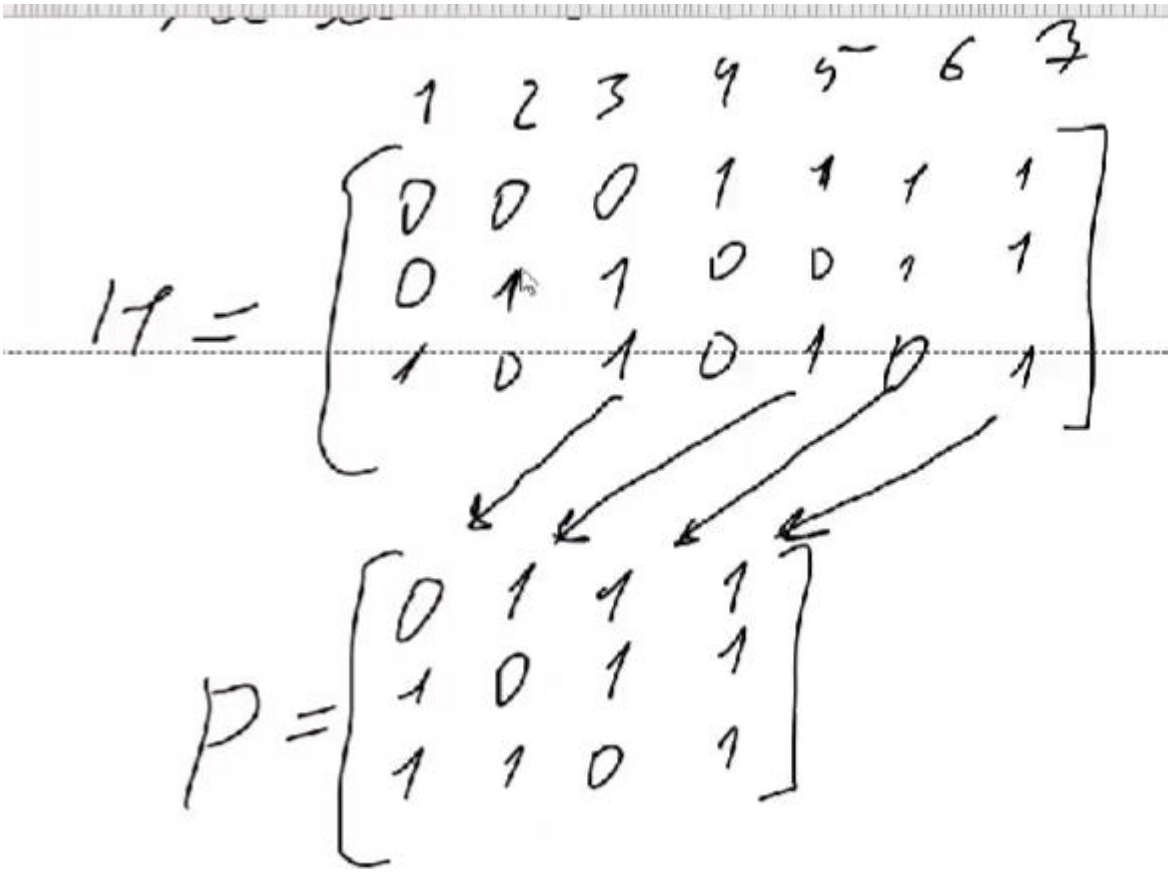
k = 3 r = 3 n = 7

Теперь мы выстраиваем матрицу H. Каждый столбец будет двоичным представлением номера столбца.

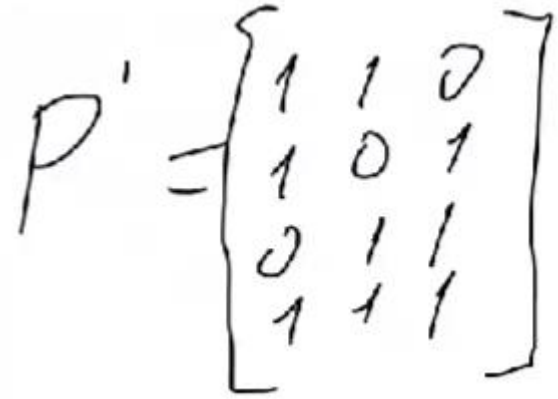


Так как это степени двойки, там всего-то одна единичка, в остальных столбцах больше единичек.

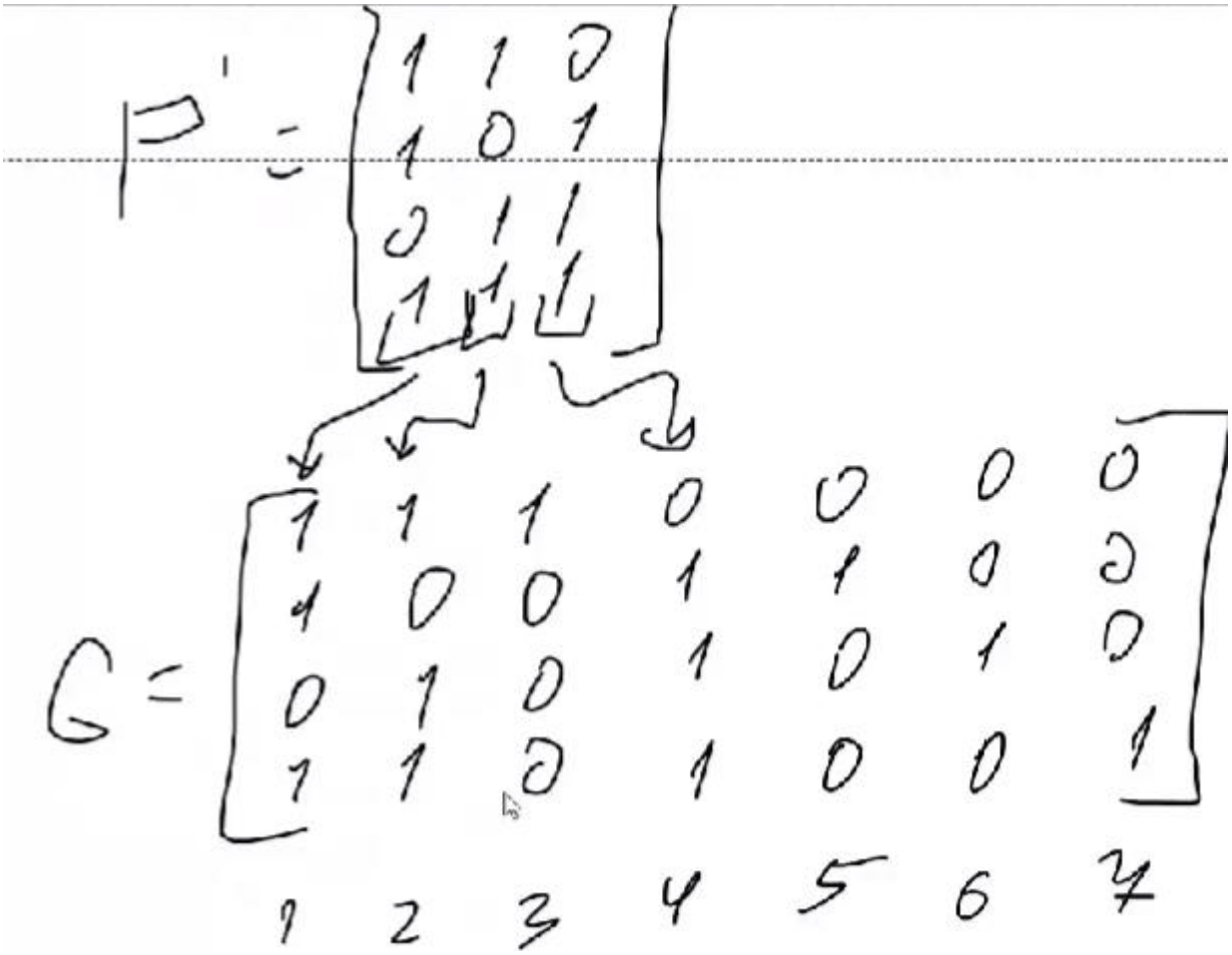
2) Выделяем проверочную подматрицу. Элементы находятся на разрядах, не являющихся степенями двойки.



3) Поворачиваем P по часовой стрелке на 90 градусов. Нет, это не транспонирование.



4) Ставим столбцы P’ на разряды с номерами, равными степеням двойки. Остальные заполняем единичной матрицей. Получаем матрицу G.



При таком алгоритме построения соблюдается ортогональность G и H, т.е. $H * G^T = 0$, $G*H^T = 0$. Рассмотрим на примере:

m = 1 0 1 0

$$G = \begin{matrix} 1110000 \\ 1001100 \\ 0101010 \\ 1101001 \end{matrix}$$

$$u = m * g = \begin{matrix} 1 \\ 0 \\ 1 \\ 0 \end{matrix} \begin{matrix} 1110000 \\ * \; 1001100 \\ 0101010 \\ 1101001 \end{matrix} = 1011010$$

u = 1 0 1 1 0 1 0

$$H = \begin{matrix} 0001111 \\ 0110011 \\ 1010101 \end{matrix}$$

$$s = H*u = \begin{matrix} 0 \\ 0 \\ 0 \end{matrix}$$

1 2 3 4 5 6 7

~u = 1 0 1 1 1 1 0

$$H = \begin{matrix} 0001111 \\ 0110011 \\ 1010101 \end{matrix}$$

$$s = H*u = \begin{matrix} 1 \\ 0 \\ 1 \end{matrix} \Rightarrow 5_{10}$$

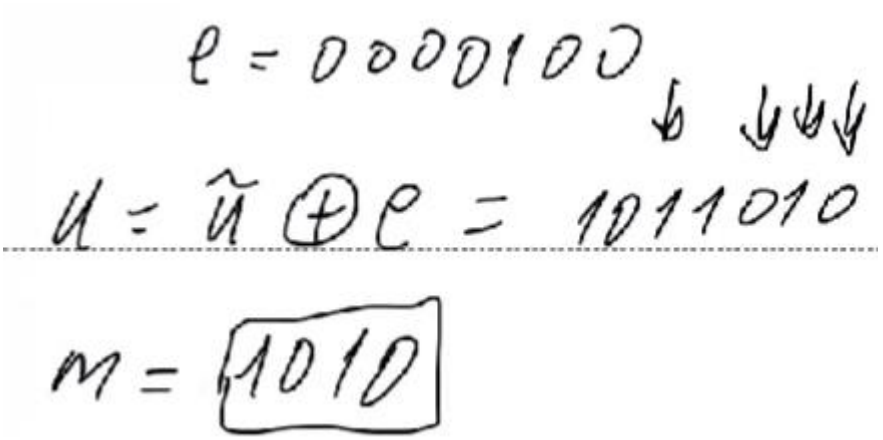


Рис. 10. Пример

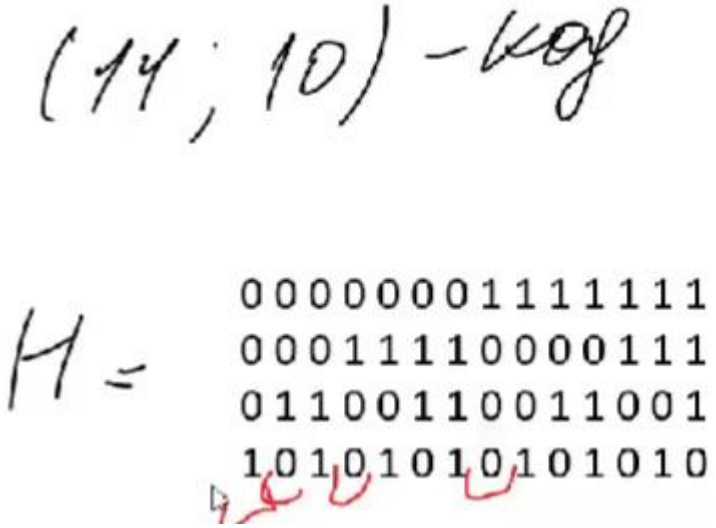
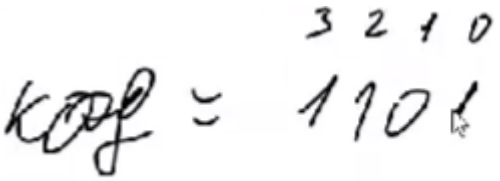


Рис. 11. Если вдруг большое число

Но есть один нюанс. R = 5, теоретически можно исправить 31 ошибку. Двукратных ошибок может быть 28, их мы можем исправить. Однако, если будут еще и однократные ошибки, то это еще 8 ошибок. В сумме 36 ошибок. Тогда мы не сможем точно утверждать, где произошла ошибка.

Циклические коды. Основная теория

Полином с фиктивной переменной x в степени n – 1.



Тут нумерация идет с конца, как в числе.

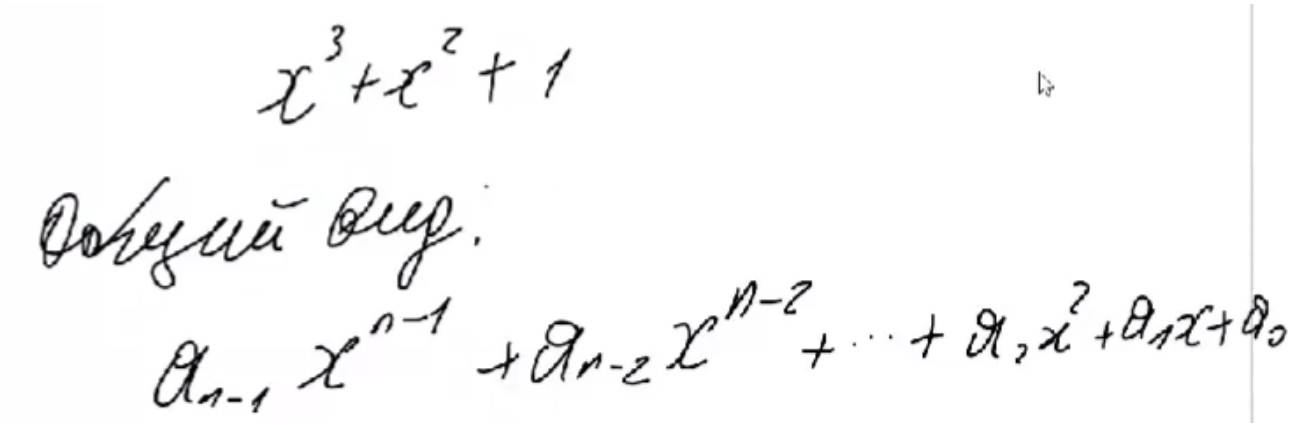
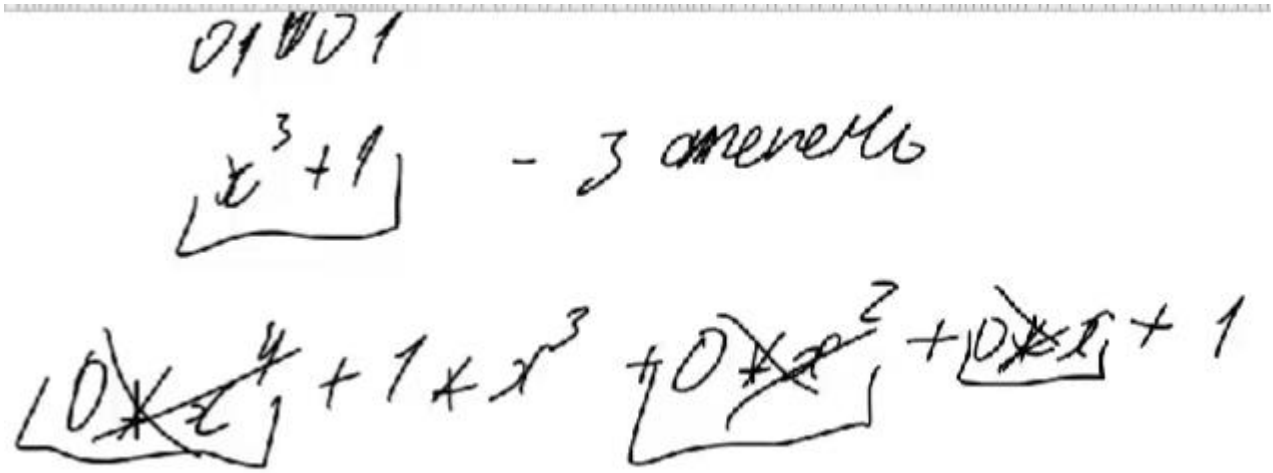
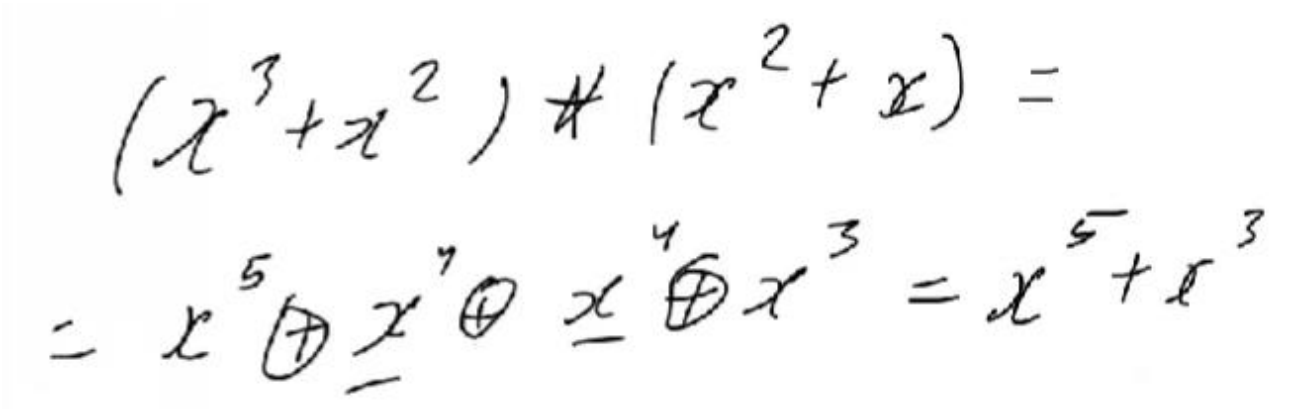


Рис. 12. Возвращаемся к информационной безопасности

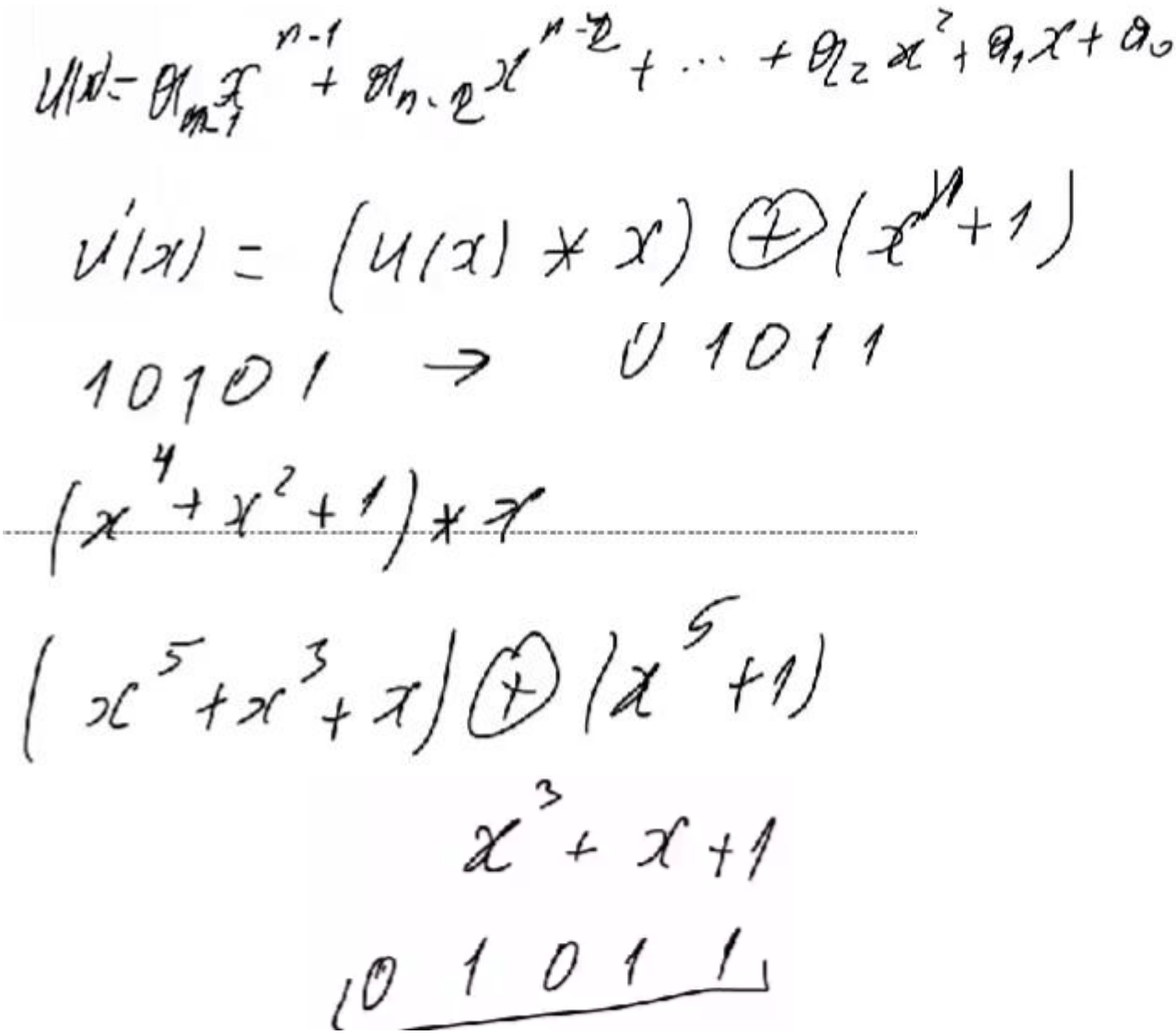
A_i – значения разрядов (нумерация справа налево). Наибольшая степень x с ненулевым коэффициентом называется степенью полинома.



Действия над полиномами: умножение, деление, сложение в поле Галуа второго порядка.



Циклический сдвиг полинома в степени N - 1 можно осуществить:



Если в качестве начального взять некоторый g(x), то можно получить разрешенные комбинации циклическим сдвигом.

$$u_1(x) = g(x)$$

$$u_2(x) = u_1(x) * x + (x^n + 1)$$

$$g(x) * x + (x^n + 1)$$

$$u_3 = u_2(x) * x + (x^n + 1) =$$

$$= g(x) * x^2 + (x^n + 1)(x + 1)$$

Ну и так далее.

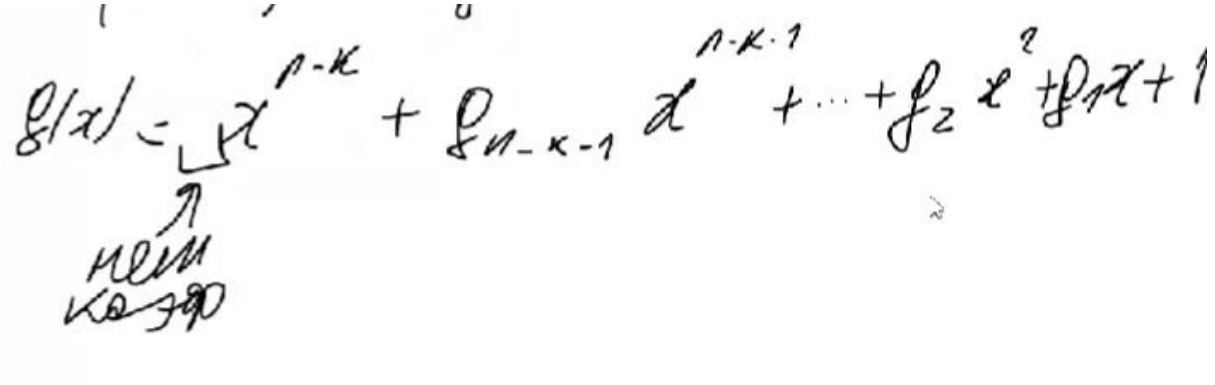
$$u_{n-1}(x) = g(x) * x^{n-1} + (x^n + 1)(x^{n-2} + x^{n-3} + \dots + 1)$$

в таком случае $g(x)$ мад-ся порождающим полиномом

Если $g(x)$ является делителем $(x^n + 1)$, то все $u_i(x)$ будут делиться на $g(x)$ без остатка

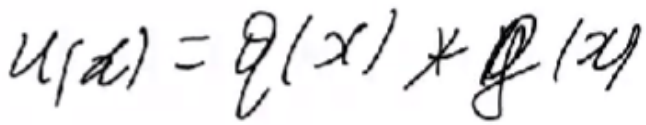
Основные свойства циклических кодов

1. В циклическом (n, k) – коде каждый кодовый полином $u_i(x)$ должен иметь степень не более n – 1.
2. Для каждого циклического кода существует собственный и единственный g(x) степени n – k вида :



Если не будет последней единицы, вы взяли не тот полином, неправильный.

3. Каждый разрешенный кодовый полином u(x) является кратным g(x), т.е.



Где q – некоторый полином.

4. g(x) является делителем $(x^n + 1)$.

Кодирование циклическим кодом

Существует два способа кодирования:

Способ № 1

Допустим, у нас есть наша k-значная комбинация, которую можно выразить в виде полинома.

$$M(x) = M_{k-1} * x^{k-1} + \dots + M_1 x + M_0$$

$$M = 1010 \qquad M(x) = x^3 + x$$

Чтобы закодировать и получить u(x) нам нужно:

$$U(x) = M(x) * g(x)$$

Здесь сложение по модулю 2, про это не забываем.

$$g(x) = x^3 + x + 1$$

$$U(x) = (x^3 + x)(x^3 + x + 1) =$$

$$= x^6 \oplus x^4 \oplus x^3 \oplus x^4 \oplus x^2 \oplus x =$$

$$= x^6 + x^3 + x^2 + x$$

$$U = 1001110$$

Здесь нет такого четкого разделения, где информационные разряды и где проверочные. Код, полученный таким образом, теряет свойства систематичности. Невозможно отделить информационную часть от проверочной.

Способ № 2

Мы тоже будем работать с полиномом, но используется деление полиномов и вычисление остатков.

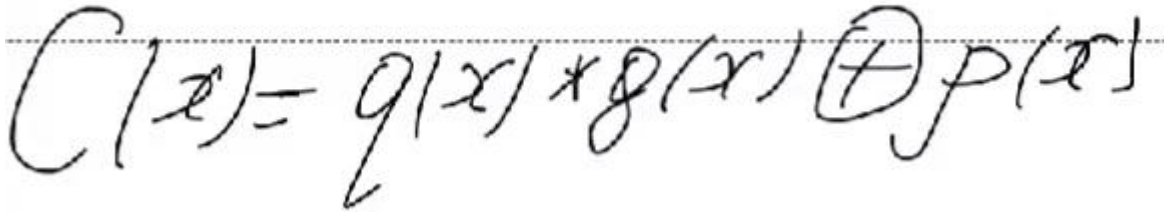
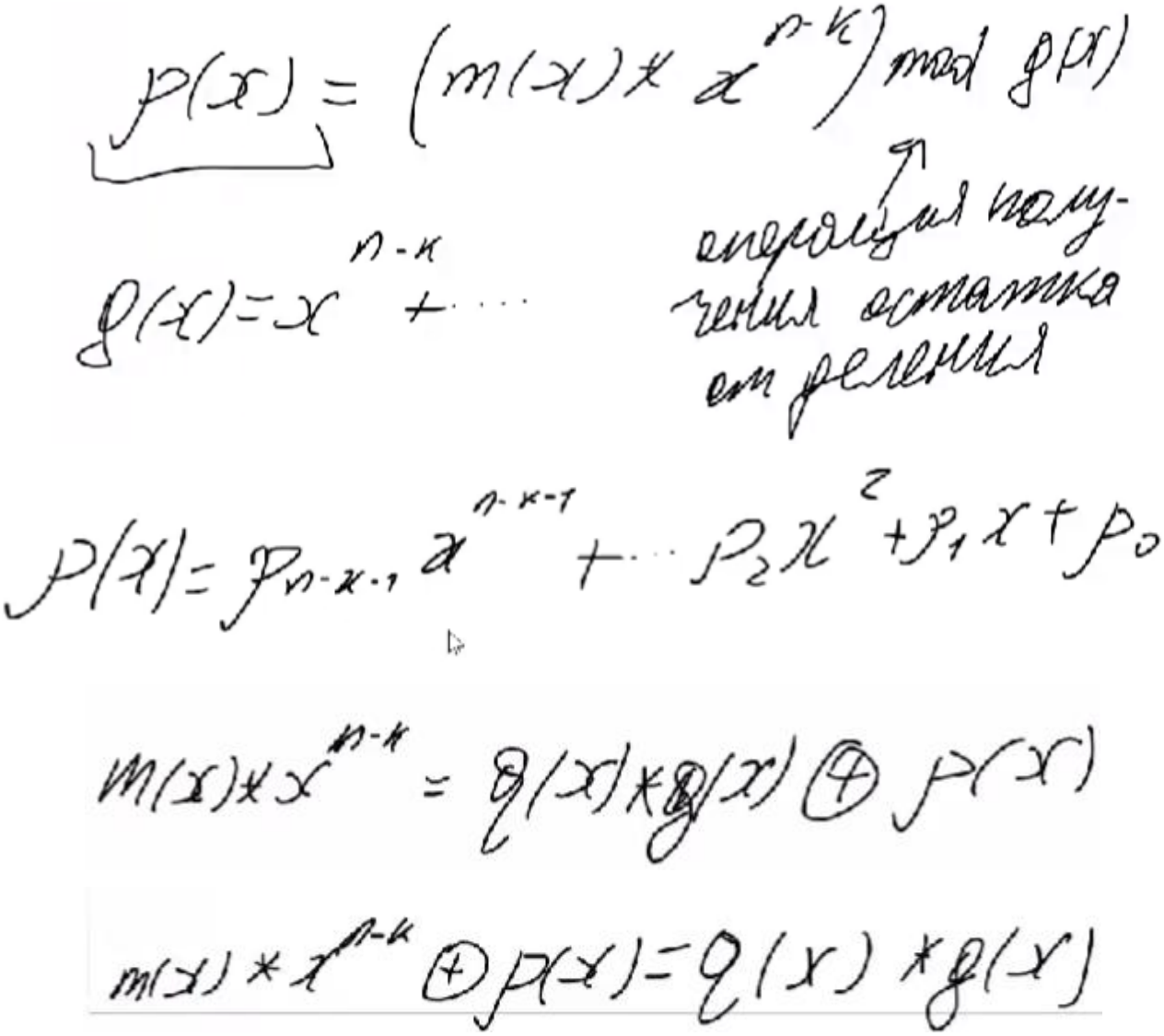


Рис. 1. C – исходная, g – делитель, q – частное, ро – остаток.

Есть m(x), g(x). Наша задача будет найти «ро».



$$m(x) * x^{n-k} \oplus p(x) = q(x) * g(x)$$

$$U(x)$$

$$u(x) = m(x) * x^{n-k} \oplus p(x) =$$

$$= m(x) * x^{n-k} \oplus \left((m(x) * x^{n-k}) \bmod g(x) \right)$$

Рис. 2. Формулы

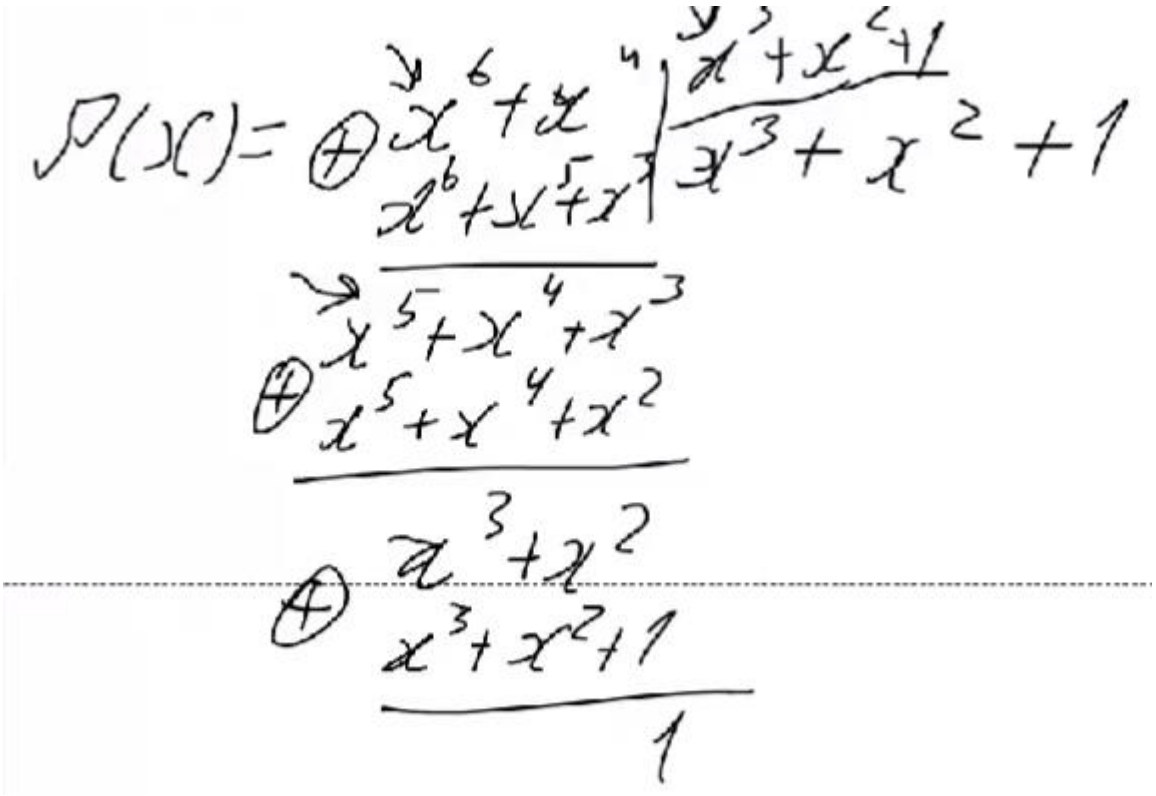
$k = 4$

$m = 1010$

$m(x) = x^3 + x$

$g(x) = x^3 + x^2 + 1$

$$m(x) * x^{7-4} = (x^3 + x) * x^3 = x^6 + x^4$$



Po(x) = 1

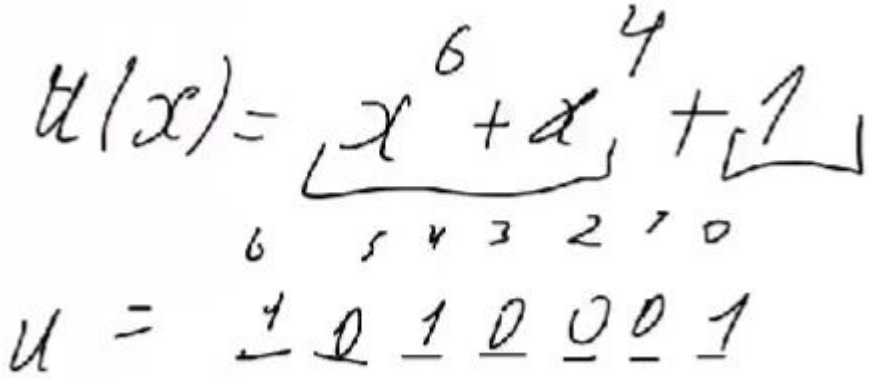


Рис. 3. Полученный помехоустойчивый код

В двоичном виде:

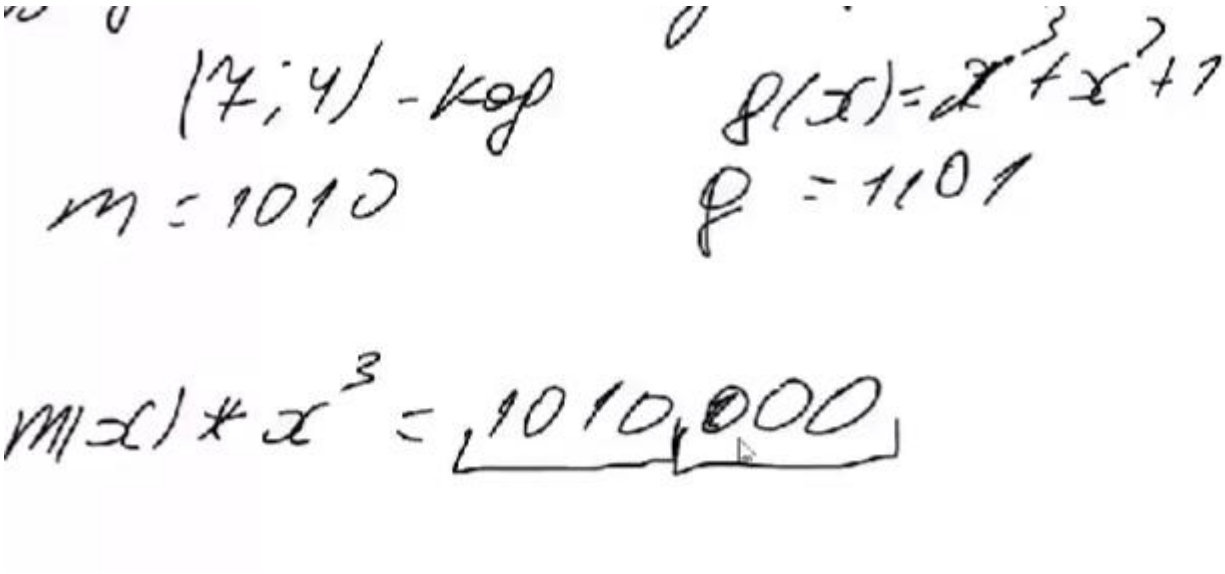


Рис. 4. При умножении мы просто сдвинули на 3 разряда влево

При делении мы просто постоянно сдвигаем.



Рис. 5. Если работать с массивом, а не с побитовыми операциями

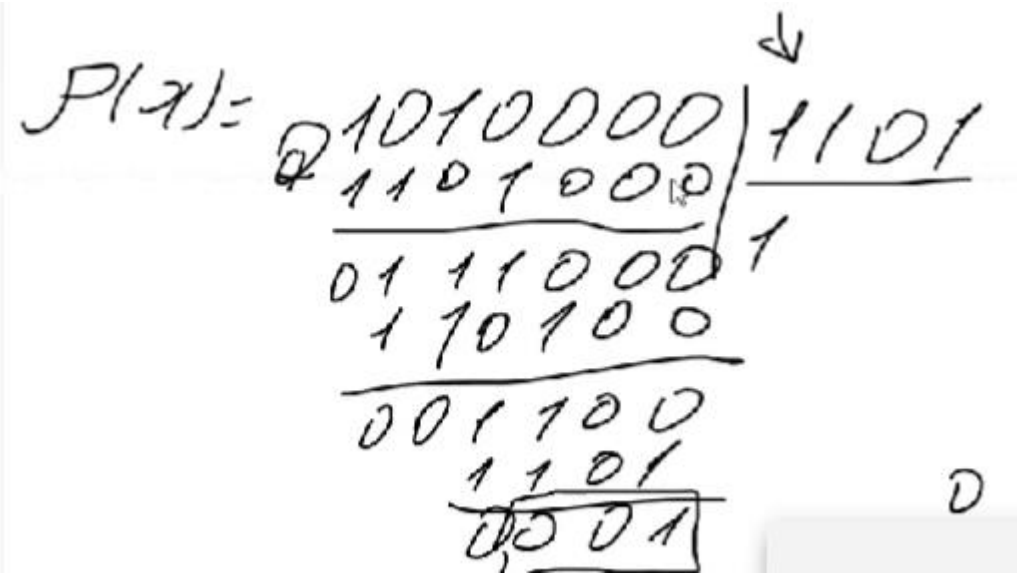
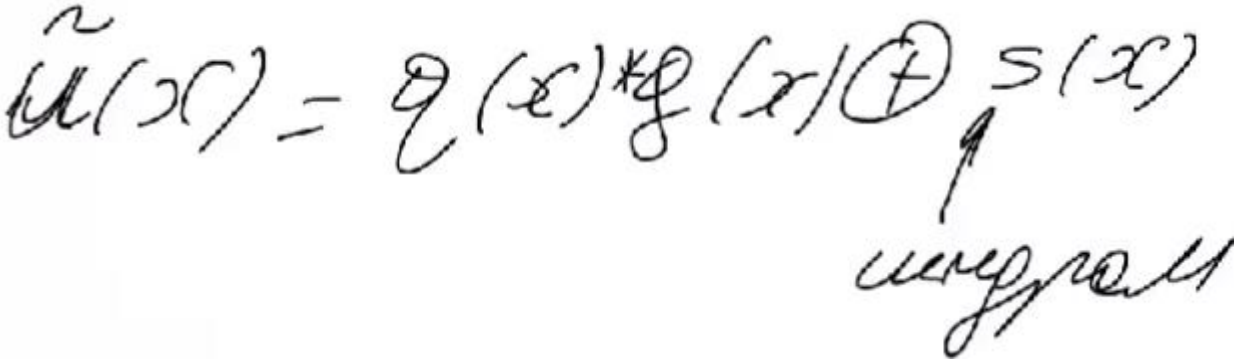


Рис. 6. При работе с побитовыми

Вычисление синдрома и исправление ошибки в циклических кодах



Если синдром равен нулю, то ошибки нет.

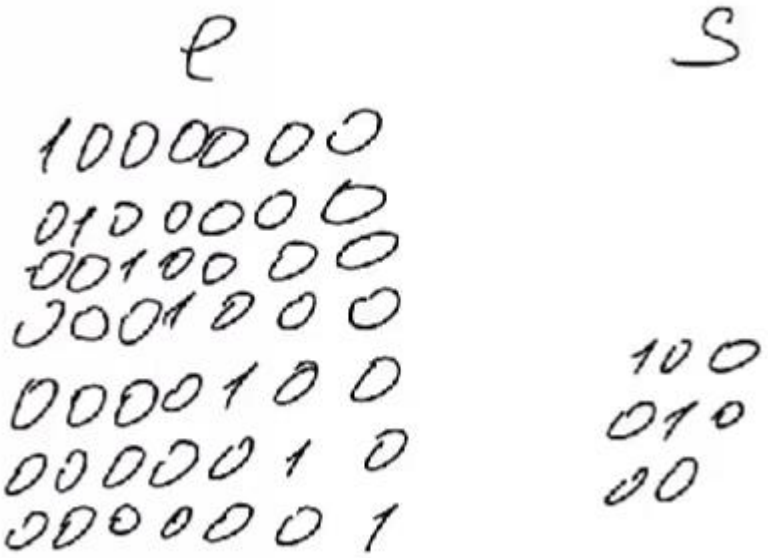


Рис. 7. Вектор ошибки и синдром

Будем искать синдромы ошибок для остальных комбинаций.

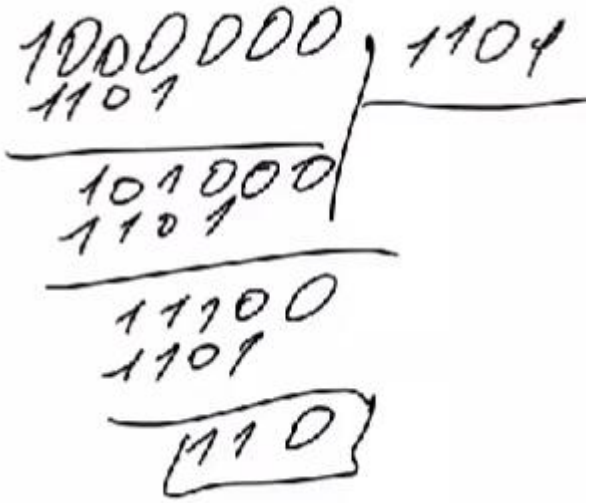


Рис. 8. Для первого разряда

И таким образом мы находим все синдромы ошибок.

